

Project 1 — readme

Name: Paavan Randhawa Student number: 301614138

Before you submit. Fill in both sections, delete these instructions, then export this document to PDF and save it as `readme.pdf` inside your `WebServer` folder. In Word: File → Save As, and choose PDF. Submit the PDF, not this `.docx` file. **Two pages maximum. The TA stops reading at the end of page two.**

1. AI use (~1 page)

AI assistance is allowed. The condition is that you understand every line you submit and can explain it if asked.

If you used no AI tools at all, delete the questions below and write this one line:

I did not use AI tools on this project.

Otherwise, answer all of them.

Which two or three prompts did the most work?

I used AI tools to generate test cases for tasks 1-5, and for help understanding the features of some of the libraries, and how they worked

What did the AI get wrong, leave incomplete, or state confidently but incorrectly?

Sometimes, the AI would leave out key features that I needed to test

What did you do instead?

I couldn't trust just the test cases that the AI gave me. I had to be specific and tell it what types of cases to test and then review the test cases it gave me to ensure those were all the cases that I wanted to test.

2. Design (~1 page)

A short paragraph for each question, about one page in total. Write about your code, not about HTTP in general.

How does your server know a request has ended?

My server recognizes that multiple requests can be received at once. Therefore, the server stores the entire received chunk in a dictionary. The key corresponds to the connected client, and the value is the request received. The server keeps calling `recv` until a “`\r\n\r\n`” is present in the message that has been received. Once it detects a “`\r\n\r\n`”, it extracts everything up until the “`\r\n\r\n`” and then returns that. Everything after the “`\r\n\r\n`” is stored in the dictionary for the next request the server will parse. At

the beginning of `recv_request_head`, the server first checks the dictionary to see if the client has sent a message already. If it has sent a message, it checks the dictionary to see if the full request is in the dictionary. If it is, it will extract it and put the leftovers (if any exist) and put them back in the dictionary.

Why does your client not simply read until the connection closes?

The client does not read until the connection closes, because multiple responses could be sent over the same connection since we are using persistent HTTP. If it keeps reading until the connection closes, it will read multiple responses. Therefore, it receives a response, parses through the response, stores the headers in a dictionary, and then checks content-length from the dictionary. Content length tells the client how many bytes to read from the body. The client then reads the number of bytes specified in content-length. If it keeps reading until the connection closes, it may read multiple responses from other requests (if any other requests were sent from the client). So, the client might read past the end of the current response body to the start of the next response. However, we only want to read the current response. So, we have to get the number of bytes that the current response has in the body and only read that specified amount. The bytes that come after the end of the current body have to be stored somewhere, so we do not lose them, so they are stored in the pending variable. Pending stores the bytes after the body so when we need to read the next response, we can start with the bytes in pending. It is continuously updated as we read the responses received from the server.

How does your thread pool bound concurrency — and what did you observe when you tested it?

My thread pool bounds concurrency by creating a fixed number of worker threads (the amount specified in the command line argument). The main thread creates a certain amount of worker threads based on the input entered on the command line and then begins to accept connections and place them in a queue. Then, the workers will take connections from the queue, so that multiple connections can be served at the same time. If there are x workers and n connections, x connections are able to be served at a time. When a worker thread is finished handling a request, it goes back to the queue to serve the next connection. If there are more connections than workers, the extra connections will have to wait in the queue until a worker is able to serve them. Once a worker thread takes a connection from the queue, it calls `handle_connection` to serve the request. Once it has served the request, it uses a lock to increment the variable representing the number of requests served and then prints served y (requests) to `stderr`. When I tested it, the workers successfully served all the requests, and the counter prevented race conditions and increased sequentially. This showed that the server was able to successfully handle multiple concurrent clients using the worker pool.

How this document is marked (not counted in the page limit)

5 marks for each section, awarded 0, 3 or 5.

	0	3	5
--	---	---	---

AI use	absent	present but generic — could describe any project	specific: real prompts, a real mistake, what you did about it
Design	absent, or plainly not about your own code	broadly right, thin	correct and specific about your own code, not about HTTP in general

This document is worth 10 of the project's 100 points. If it is missing from your submission, it scores 0 on this item and affects nothing else — the automated marks for the five tasks are unaffected.

You may also be picked at random for a short oral check-in and asked to walk through your own code. The design questions above are the questions you will be asked, so answering them honestly here is the preparation.